

Beyond iperf: Building a Zero-syscall Benchmark with io_uring

Overcoming Kernel Context Switch Bottlenecks and Saturation Limits on 25G/100G+ Ethernet

Presenter: Thilak Raj Surendra Babu

Reference Project: [\[trajs/uring_perf\]](#)

Traditional POSIX

Userspace syscall to kernel (~1.2M syscalls/sec)
[BOTTLENECK: KERNEL CONTEXT SWITCHING]

Modern io_uring

Userspace to NIC via shared rings (0 syscalls via SQPOLL)
[OPTIMIZED: ZERO-SYSCALL SATURATION]

Iperf VS MODERN SPEEDS

■ SYS // KERNEL // IO_URING

[BENCHMARKING // IPERF3 BOTTLENECKS]

The Standard Tool Meets Modern Wire Speeds

// 01: LEGACY FOUNDATION

Gold standard for performance validation, Easy to setup, use . Has what most people want

// 02: HIGH-SPEED SCALE CHALLENGE

- **25 Gbps:** Requires 3.125 GB/s payload (~2.1M packets/sec).
- **100 Gbps:** Demands 12.5 GB/s payload (~8.2M packets/sec).

// 03: SINGLE-THREAD BOTTLENECK

Single-threaded event loop hits a hard ceiling at **18-23 Gbps** (1500 MTU) due to per-packet system call costs and POSIX memory copies.



The Per-Packet & Per-Byte Tax of POSIX

■ SYS // KERNEL // IO_URING

// 01: CONTROL PLANE OVERHEAD

The Per-Packet Syscall Tax

Traditional benchmarking tools fail at line rate due to the microarchitectural tax imposed by standard POSIX write and send paths.

Context Switches: Every operation forces user-to-kernel context switches (Ring 3 → Ring 0). Exacerbated by Spectre/Meltdown mitigations including **KPTI**, **retpolines**, and **IBPB**.

Kernel Execution: Inside the kernel, execution flows through `sock_sendmsg()`, allocates Socket Buffers (SKBs), and calls `copy_from_user()`.

~195,000

SYSCALLS / SEC REQUIRED

Unsustainable frequency required to saturate a 100 Gbps link using 64KB chunks.

// 02: DATA PLANE BOTTLENECK

The Per-Byte Memory Copy Tax

Beyond control plane context switching, the physical memory copy tax inflicts devastating damage on the system bus architecture.

Bus Saturation: Executing `copy_from_user()` to move 12.5 GB/s of network data twice consumes a staggering **25 GB/s** of DDR4/DDR5 bandwidth.

CPU Starvation: Massive data movement rapidly triggers memory controller bottlenecks, starving the CPU of compute cycles long before reaching maximum NIC capacity.

25 GB/s

DDR4 / DDR5 BUS BANDWIDTH CONSUMED

Moving 12.5 GB/s twice exhausts memory controllers before hardware line rate is hit.

RSS HASHING, CORE AFFINITY & QUEUE CONTENTION

■ SYS // KERNEL // IO_URING

[01 // HARDWARE QUEUES]

Hashing inconsistency

Multiple iperf flows frequently experience collisions, mapping to the exact same RX queue and CPU core, causing severe SoftIRQ saturation while adjacent processing units remain idle.

[02 // MEMORY DOMAINS]

NUMA Alignment Penalty

iperf3 lacks native thread-to-core pinning and NUMA awareness. When memory is allocated on one NUMA node but the network stack is processed on another, cross-NUMA interconnect transfers heavily degrade overall throughput.

[03 // IMPACT METRIC]

30-40%

THROUGHPUT PENALTY

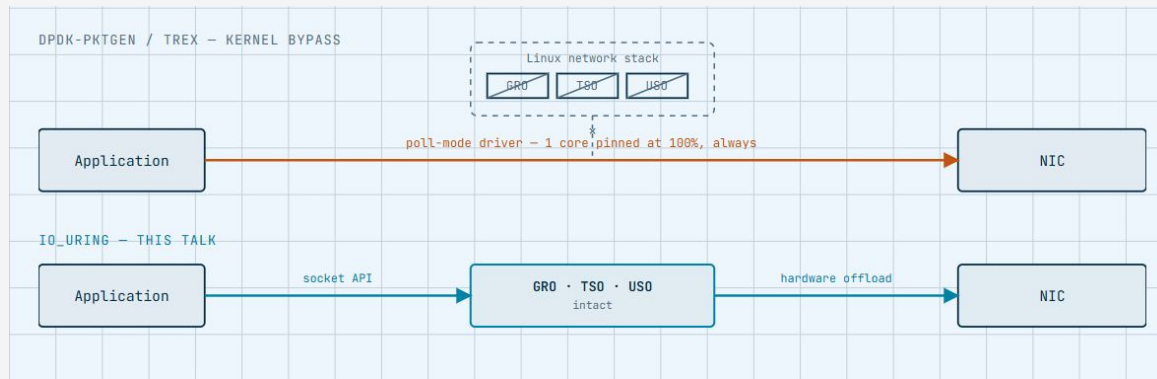
Severe performance degradation resulting directly from cross-NUMA memory transfers and unpinned thread execution.

// ARCHITECTURAL DIAGNOSTIC FAILURE

Running multi-flow iperf (-P) to bypass single-thread limits frequently fails at scale due to these deep architectural mismatches. Ultimately, parallel streams in traditional tools evaluate the statistical luck of RSS hash distribution rather than true kernel stack and hardware capability.

Alternatives(Kernel Bypass/Multiprocess)

■ SYS // KERNEL // IO_URING



01 // MULTI-PROCESS IPERF

Involves scripting multiple independent iperf3 instances bound to separate ports to increase aggregate bandwidth.

CRITICAL OVERHEAD

Requires complex orchestration and still suffers from identical high system call and memory copy overheads.

02 // KERNEL BYPASS (DPDK)

Uses tools like Pktgen or TReX with poll-mode drivers to easily achieve 100G/200G line rates with zero syscalls.

STACK BLINDNESS

Bypasses Linux kernel stack—ignoring Netfilter, eBPF, Cgroups, and TCP congestion control.

Complex Setup: Hugepages, DPDK driver binding.

03 // IO_URING (OPTIMAL)

Utilizes tools like uring_perf to test the actual Linux TCP stack with zero syscalls and zero memory copies.

PRODUCTION FIDELITY

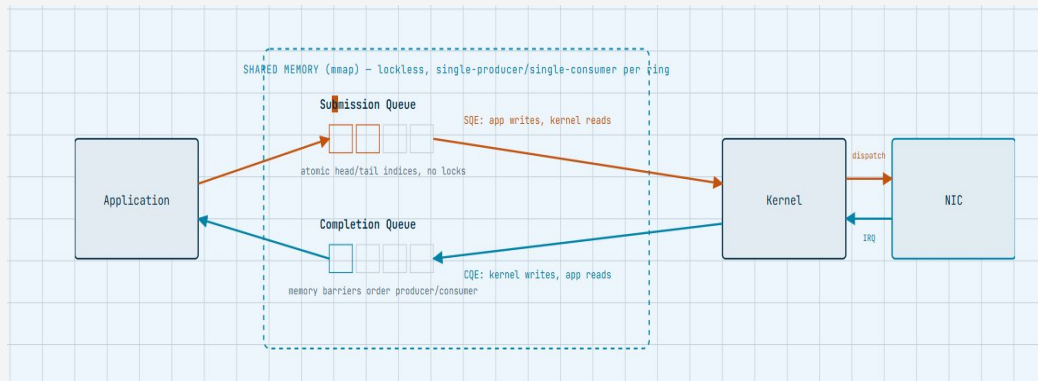
Operates seamlessly alongside standard Linux diagnostic and security tools, accurately reflecting real host network performance.

THE PARADIGM SHIFT IN LINUX ASYNCHRONOUS I/O

■ SYS // KERNEL // IO_URING

// KERNEL 5.1 INNOVATION BY JENS AXBOE

Designed to resolve performance limits of legacy POSIX AIO. Built upon two lockless circular ring buffers in shared kernel memory mapped via `mmap()`, eliminating CPU privilege transitions from the hot path.



01 // SUBMISSION QUEUE (SQ)

User to Kernel Path

The application writes I/O requests (SQEs) directly to this shared memory ring, queuing requests without context switches or kernel privilege level changes.

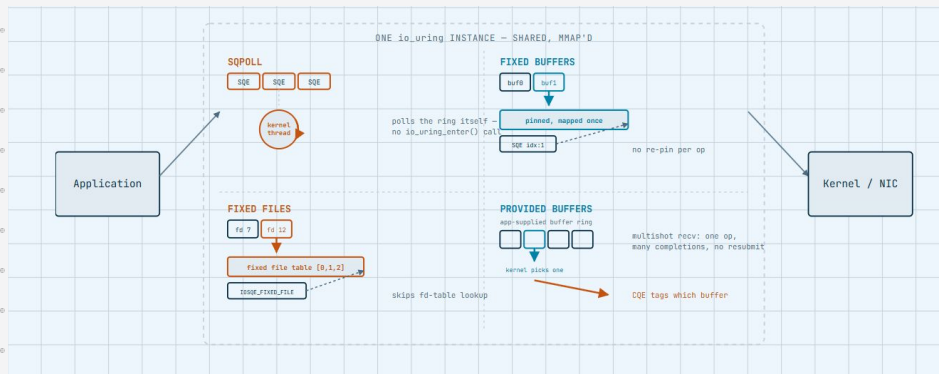
02 // COMPLETION QUEUE (CQ)

Kernel to User Path

The Linux kernel posts I/O results (CQEs) back to shared memory, enabling true zero-syscall operation for high-performance network and disk I/O.

BUILDING BLOCKS FOR MAXIMUM THROUGHPUT

■ SYS // KERNEL // IO_URING



01 // SQPoll (KERNEL POLLING)

Deploys a dedicated kernel kthread to continuously poll the Submission Queue. Submits I/O without `io_uring_enter()` syscalls.

02 // REGISTER BUFFERS

Pre-pins memory pages via `get_user_pages()`. Eliminates per-I/O page table walks and prevents TLB invalidation.

03 // REGISTER FILES

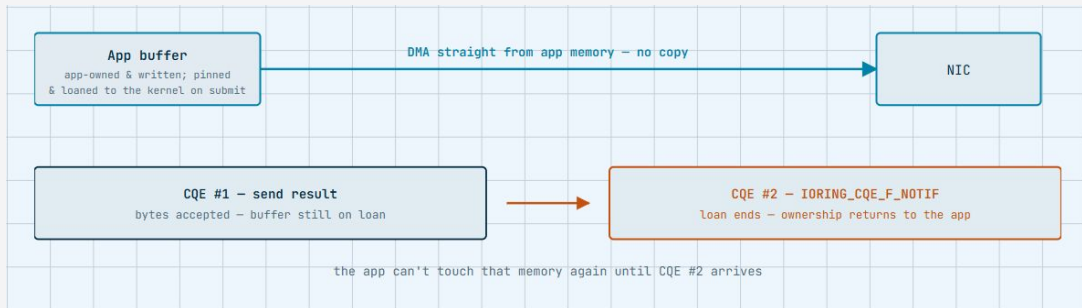
Pre-registers socket FDs in fixed file table, avoiding lock contention from `fget()` and `fput()`.

04 // PROVIDED BUFFERS

Automatic buffer assignment for incoming receives, ensuring memory management never blocks the data plane.

DRIVER & KERNEL PLUMBING ON TX PATH

■ SYS // KERNEL // IO_URING



// STAGE 01: INITIAL SUBMISSION

Kernel Memory Pinning & CQE 1

Zero-copy transmission ([IORING_OP_SEND_ZC](#), Linux 6.0+) eliminates CPU-driven data movement by pinning userspace memory pages directly into sk_buff fragment arrays.

The kernel emits an initial **Completion Queue Entry (CQE 1)** signaling that the request was queued into the kernel TCP stack.

// STAGE 02: HARDWARE DMA & NOTIFICATION

PCIe Direct Access & CQE 2

The NIC fetches payload bytes directly from userspace memory via PCIe Direct Memory Access (DMA), bypassing CPU buffers entirely.

A second notification (**CQE 2** with [IORING_CQE_F_NOTIF](#)) is emitted after physical DMA completion to allow memory reuse.

// HARDWARE & DRIVER PREREQUISITES

This two-step completion lifecycle and zero-copy operation requires modern high-performance network drivers with explicit support for scatter-gather I/O and page recycling.

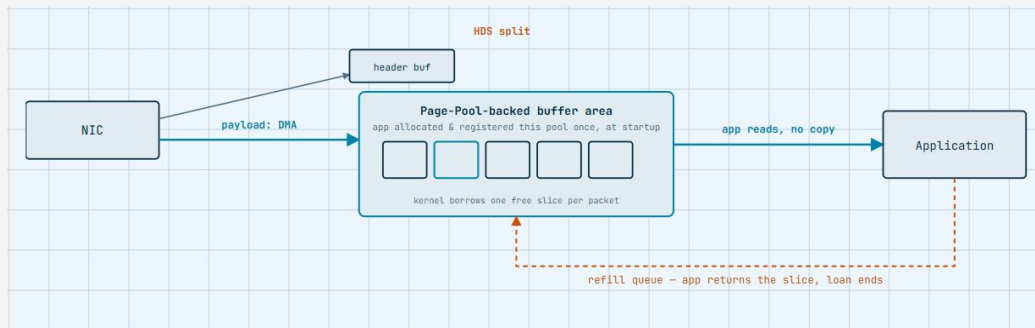
SUPPORTED DRIVERS: `mlx5` | `ice` | `i40e` | `bnxt_en`

DRIVER CHANGES & RX KERNEL SUPPORT

■ SYS // KERNEL // IO_URING

// ARCHITECTURAL CHALLENGE (RX ZERO-COPY)

Implementing zero-copy on the receive path requires parsing incoming network headers before determining socket destination. Linux 6.10+ addresses this via silicon-level Header-Data Split (HDS) and specialized stack mechanics.



[01] SILICON SPLIT

Header-Data Split (HDS)

- Supported in Linux 6.10+ via NIC hardware automation.
- Splits 54-byte Ethernet, IP, & TCP headers into kernel memory.
- Directs bulk payload into pre-registered user buffers.

[02] MEMORY ALLOCATION

Page Pool & Dmabuf TCP

- Integrated with Page Pool & Dmabuf TCP infrastructure.
- Allocates memory directly from userspace or DMABUFs.
- Feeds user memory into NIC RX hardware descriptors.

[03] FLOW STEERING

ethtool Ntuple Rules

- Deploys flow steering rules to guide benchmark traffic.
- Directs specific flows into dedicated hardware queues.
- Bypasses CPU interrupt distribution bottlenecks.

uring_perf Architecture

■ SYS // KERNEL // IO_URING

[01] MULTI-THREADING

Shared-Nothing Model

uring_perf is a modern C++17 benchmark tool engineered natively on liburing to demonstrate the absolute limits of the Linux network stack.

Each worker thread is entirely autonomous, owning an isolated io_uring instance, dedicated memory buffers, and exclusive socket descriptors, ensuring zero mutexes exist on the critical hot path.

[02] HARDWARE AFFINITY

CPU & NUMA Pinning

To maximize hardware efficiency, uring_perf enforces aggressive CPU and NUMA pinning.

Worker threads and their corresponding SQPOL kernel threads are strictly bound to logical cores local to the Network Interface Card's specific NUMA node.

[03] SOCKET SCALING

SO_REUSEPORT Architecture

The server architecture leverages the SO_REUSEPORT socket option, allowing the kernel to natively hash incoming network connections directly to the autonomous workers.

// REAL-TIME TELEMETRY & METRICS

Throughout execution, the tool provides real-time telemetry, delivering highly accurate statistics on Gigabits per second (Gbps), Millions of packets per second (Mpps), and detailed zero-copy hardware completion metrics.

HOW URING_PERF TRANSMITS AT LINE RATE

■ SYS // KERNEL // IO_URING

```
// Zero-Syscall Transmission Hot Loop
while (1) {
    sqe = io_uring_get_sqe(ring);
    io_uring_prep_send_zc(sqe, fd, buf, len, 0, IORING_RECVSEND_FIXED_BUF);
    io_uring_submit(&ring); // Under SQPOLL, acts purely as a memory barrier
    harvest_completions_from_cq(&ring);
}
```

[01 // MEMORY ALLOCATION & REGISTRATION]

All transmission buffers are allocated using `posix_memalign()` on strict 4KB boundaries and are pre-registered with the kernel via `IORING_REGISTER_BUFFERS`.

[02 // CQE HANDLING & DMA RECYCLING]

Application logic distinguishes the initial send completion (bytes accepted) from the buffer-release completion flagged `IORING_CQE_F_NOTIF`— the kernel's signal that it's truly done referencing that memory — and recycles buffers only once the second CQE arrives.

SEND PATH: IPERF3 VS. URING_PERF (WITH & WITHOUT SEND_ZC)

■ SYS // KERNEL // IO_URING

Sender-side CPU only – same rig, single TCP stream, receiver left on a plain (non-zero-copy) recv throughout

iperf3 (baseline sender)

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **37.8%**
- Cores: 1 Core

// BASELINE

Standard POSIX socket transmission baseline.

uring_perf plain send

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **30.0%**
- Flag: (no -Z)

// KERNEL COPY

Copies into the kernel, same data copying mechanism as iperf3.

uring_perf SEND_ZC (-Z)

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **4.7%**
- Efficiency: **~9× less CPU**

// ZERO-COPY OPTIMIZED

~9× less CPU than plain send with identical throughput.

RECEIVE PATH: IPERF3 VS. URING_PERF, WITH AND WITHOUT ZCRX

■ SYS // KERNEL // IO_URING

Receiver-side CPU only – same rig, single TCP stream, sender left on a plain (non-zero-copy) send throughout so only the receive path changes

iperf3 (baseline receiver)

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **81.3%**
- CPU (% of one core)

// BASELINE

Standard POSIX socket reception baseline.

uring_perf plain recv

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **65.5%**
- Standard Buffers

// KERNEL COPY

multishot + provided buffers, still copies

uring_perf zcrx (-Y)

THROUGHPUT: 23.5 Gbit/s

- CPU Load: **18.7%**
- Efficiency: **~3.5× less CPU**

// ZERO-COPY OPTIMIZED

~3.5× less CPU than plain recv, same throughput

UDP: IPERF3 VS. URING_PERF, SENDER AND RECEIVER CPU TOGETHER

■ SYS // KERNEL // IO_URING

Same rig, unlimited-rate UDP flood – each pair of bars is one config's sender (TX) and receiver (RX), on a true 0–100% scale

iperf3

CPU LOAD (SENDER / RECEIVER)

- TX (Sender): 67.7%
- RX (Receiver): 100.0%

THROUGHPUT

sends 8.3, only 5.9 Gbit/s
arrives

// HIGH LOSS baseline

29% PACKET LOSS

Receiver saturated at 100% CPU.

uring_perf plain

CPU LOAD (SENDER / RECEIVER)

- TX (Sender): 41.7%
- RX (Receiver): 63.8%

THROUGHPUT

23.9 Gbit/s

// ZERO LOSS

0% LOSS

Full line-rate achieved via io_uring batching.

uring_perf SEND_ZC (-Z)

CPU LOAD (SENDER / RECEIVER)

- TX (Sender): 4.0%
- RX (Receiver): 55.8%

THROUGHPUT

23.9 Gbit/s

// ZERO-COPY OPTIMIZED

0% LOSS • 10× LESS TX CPU

Sender CPU load drops from 41.7% to 4.0%.

EXPANDING IO_URING BENCHMARKING

■ SYS // KERNEL // IO_URING

`feature_01 // kernel_6.12+`

FULL BIDIRECTIONAL ZERO-COPY (ZC_RX)

Fully integrates Linux 6.12+ Dmabuf TCP and zc_rx APIs into uring_perf, allowing line-rate payload reception without CPU intervention.

`feature_02 // multi_flow_scaling`

SUPPORT FOR MULTIPLE FLOWS

Scales across concurrent TCP/UDP connections to measure total aggregated throughput and per-flow latency distribution across multi-core systems.

`feature_03 // high_throughput`

QUIC PROTOCOL SUPPORT

Expands protocol suite to include QUIC, benchmarking complex transports at 100G/200G using UDP_SEGMENT hardware offloads.

`feature_04 // zero_syscall_observability`

EBPF TELEMETRY ENGINE INTEGRATION

Integrates a low-overhead eBPF engine hooking into tcp_probe to extract granular congestion window (cwnd) and RTT metrics without perturbing the sensitive hot path.

RUNNING URING_PERF: FLAGS & COMMANDS

■ SYS // KERNEL // IO_URING

Same binary throughout – every mode is a flag, not a different tool

// ONE-TIME HARDWARE SETUP – ONLY NEEDED FOR -Y / ZCRX

steer one TCP port to a dedicated hardware Rx queue, then hand that queue to zcrx

```
$ ethtool -G eth0 tcp-data-split on
```

```
$ ethtool -N eth0 flow-type tcp4 dst-port 9100 action 1
```

// RUNNING THE TOOL

```
$ uring_perf -s -p 5201 # server, plain TCP
```

```
$ uring_perf -c 10.0.0.2 -p 5201 -t 10 # client, run for 10s
```

```
$ uring_perf -c 10.0.0.2 -p 5201 -Z # zero-copy SEND
```

```
$ uring_perf -s -p 9100 -Y --zcrx-if eth0 --zcrx-queue 1 # zero-copy RECEIVE (needs setup above)
```

```
$ uring_perf -c 10.0.0.2 -p 5201 -S --sqpoll-cpu 2 -A 1 # SQPOLL, pinned
```

```
$ uring_perf -s -u -P 4 -M # UDP, 4 threads, one process each
```

// VERIFICATION & DRIVER REQUIREMENTS

-Y additionally needs a driver with zcrx queue-restart support

THE FUTURE OF LINUX NETWORK BENCHMARKING

■ SYS // KERNEL // IO_URING

// CORE FINDING & PARADIGM SHIFT

"uring_perf proves that the standard Linux TCP stack can effortlessly saturate 100 Gbps wire speeds on a single application flow without exhausting CPU resources."

// LEGACY TOOL LIMITATIONS & PITFALLS

- **iperf3 Bottlenecks:** Crippled by system call overhead, memory copies, and a 25 Gbps single-thread ceiling.
- **DPDK Pitfalls:** Completely bypasses the Linux kernel stack, failing to test real-world application paths.
- **Multi-stream iperf:** Suffers heavily from RSS hash collisions and severe NUMA memory penalties.

// THE IO_URING ARCHITECTURAL SOLUTION

- **In-Kernel Execution:** Leverages SQPOLL for a true zero-syscall asynchronous interface.
- **Zero-Copy Data Path:** Uses SEND_ZC transmission mechanism to eliminate expensive memory copies.
- **Uncompromised Testing:** Evaluates native kernel network performance accurately at full wire speed.

CALL TO ACTION // OPEN SOURCE BENCHMARK

Explore the source, review the architecture, and contribute at [\[trajs/uring_perf \]](https://github.com/trajs/uring_perf)

// DISCUSSION & QUESTIONS

Have Any Questions?

Open for questions regarding benchmark methodology, io_uring zero-copy implementations, or kernel networking performance.

- **io_uring ZC_RX / ZC_TX** configuration
- **UDP & TCP** workload saturation
- **Hardware offloads** & driver queueing

// CONTACT & RESOURCES

Get In Touch

AUTHOR

Thilak Raj

EMAIL

s.thilakraj@gmail.com